

Modelling Nuclear Fusion Reactor Power Outputs with Machine Learning Techniques

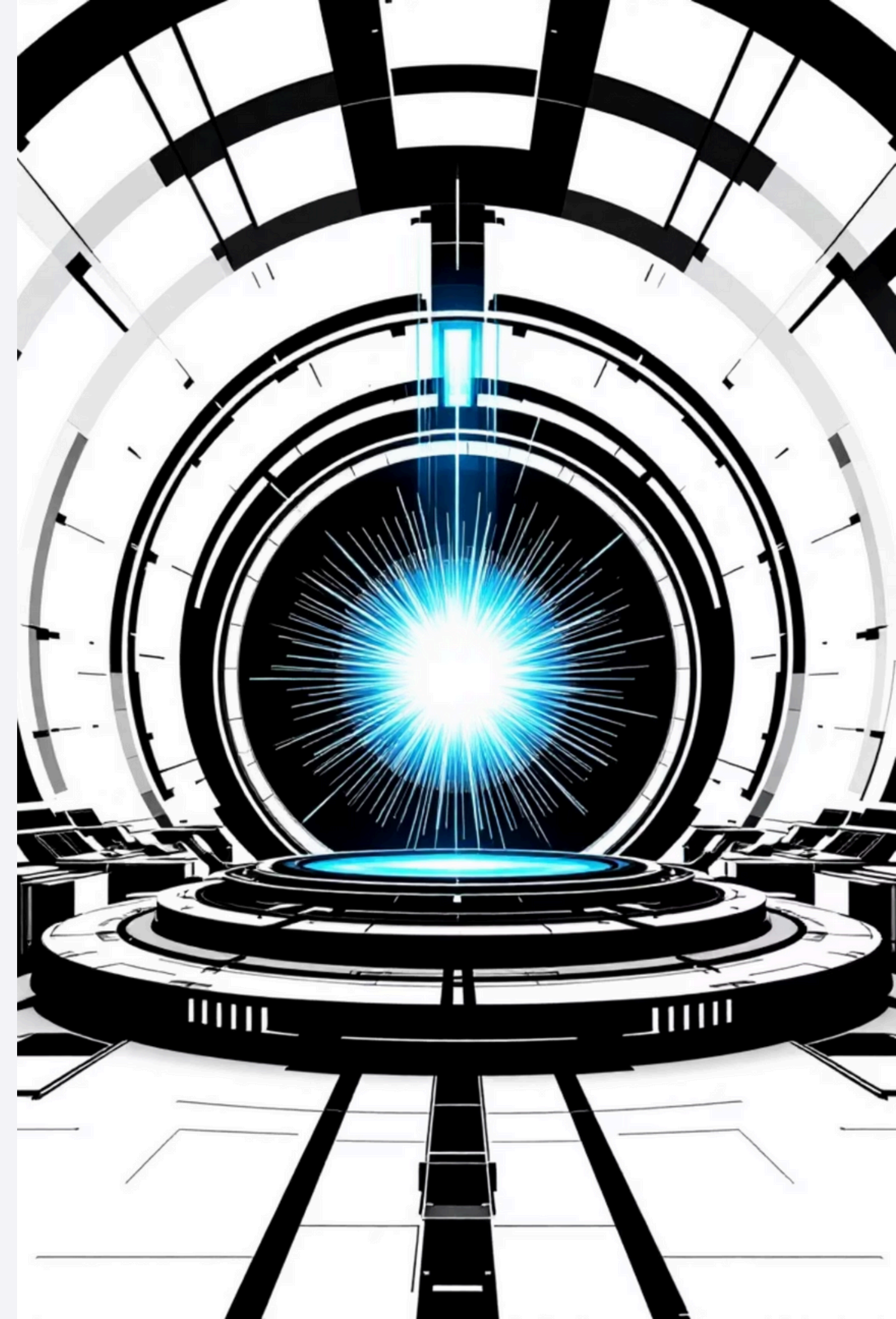
CHE657 Course Project | Group 7

Chandan Achary (230319)

Harshit Tomar (230462)

Rutul Bhanushali (230884)

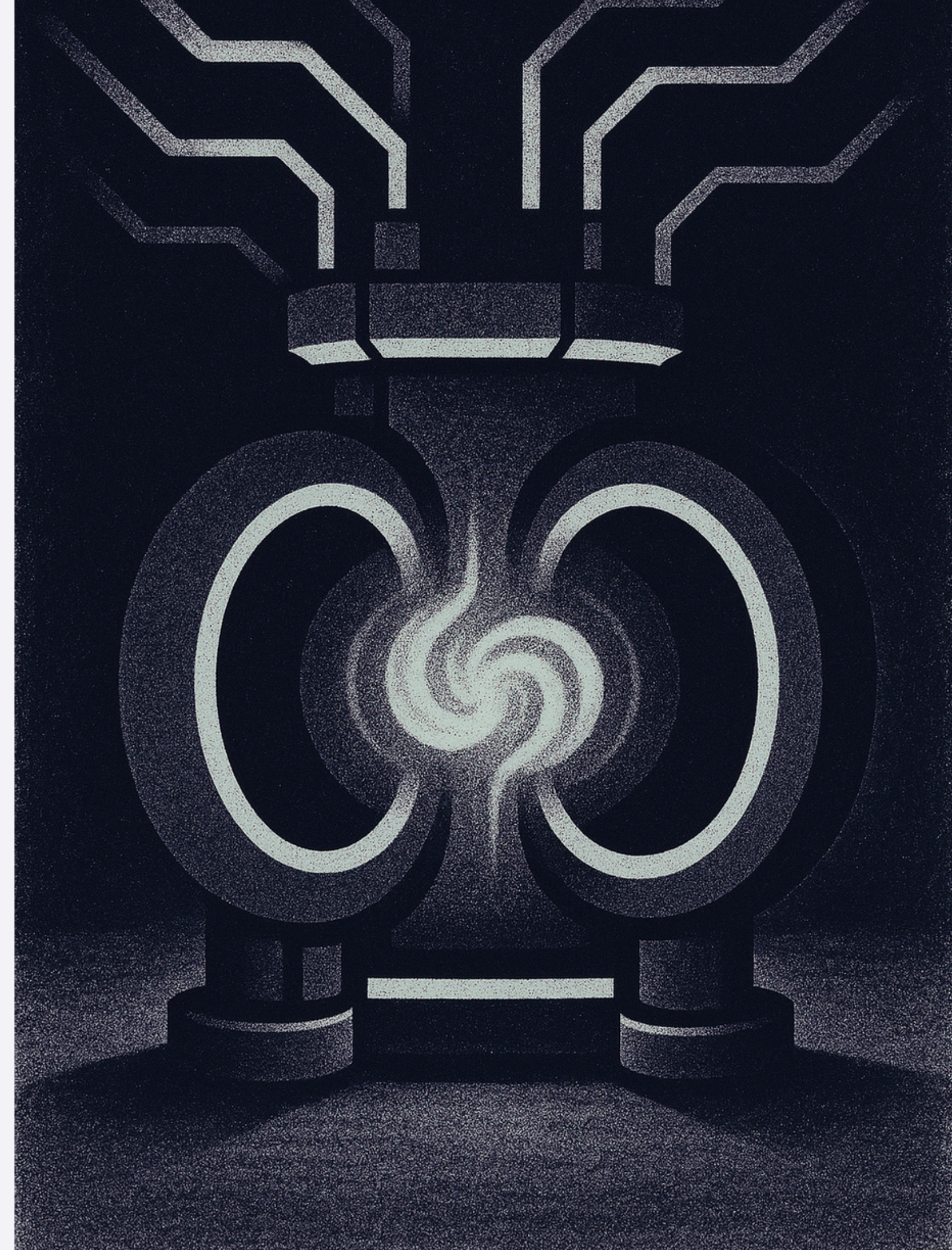
Vishal Raj (231163)



Project Overview

Motivation and Objectives

- Fusion energy is a promising clean and theoretically a virtually limitless power source.
- Currently fusion reactor designs fail to achieve net energy gain, ie, they consume more power than they produce.
- Experimental testing is extremely costly and resource-intensive, limiting the amount of real data available.
- This makes modelling and simulation a powerful alternative for exploring design and performance improvements.
- Accurately modeling fusion reactor outputs is extremely complex and computationally intensive.
- By applying machine learning, we aim to develop data-driven models that can predict and optimize power outputs efficiently.



Project Overview

– Data Collection and Processing

- Data Categorization
- Value Scalling
- Train-Test Split

– Multiple Linear Regression

- Regularization
- Hyperparameter Tuning

– Neural Network

- Built a feed-forward network to capture non-linear dependencies between plasma parameters and power output.

– Physics Informed NN

- Combined neural network predictions with physical laws of nuclear fusion.

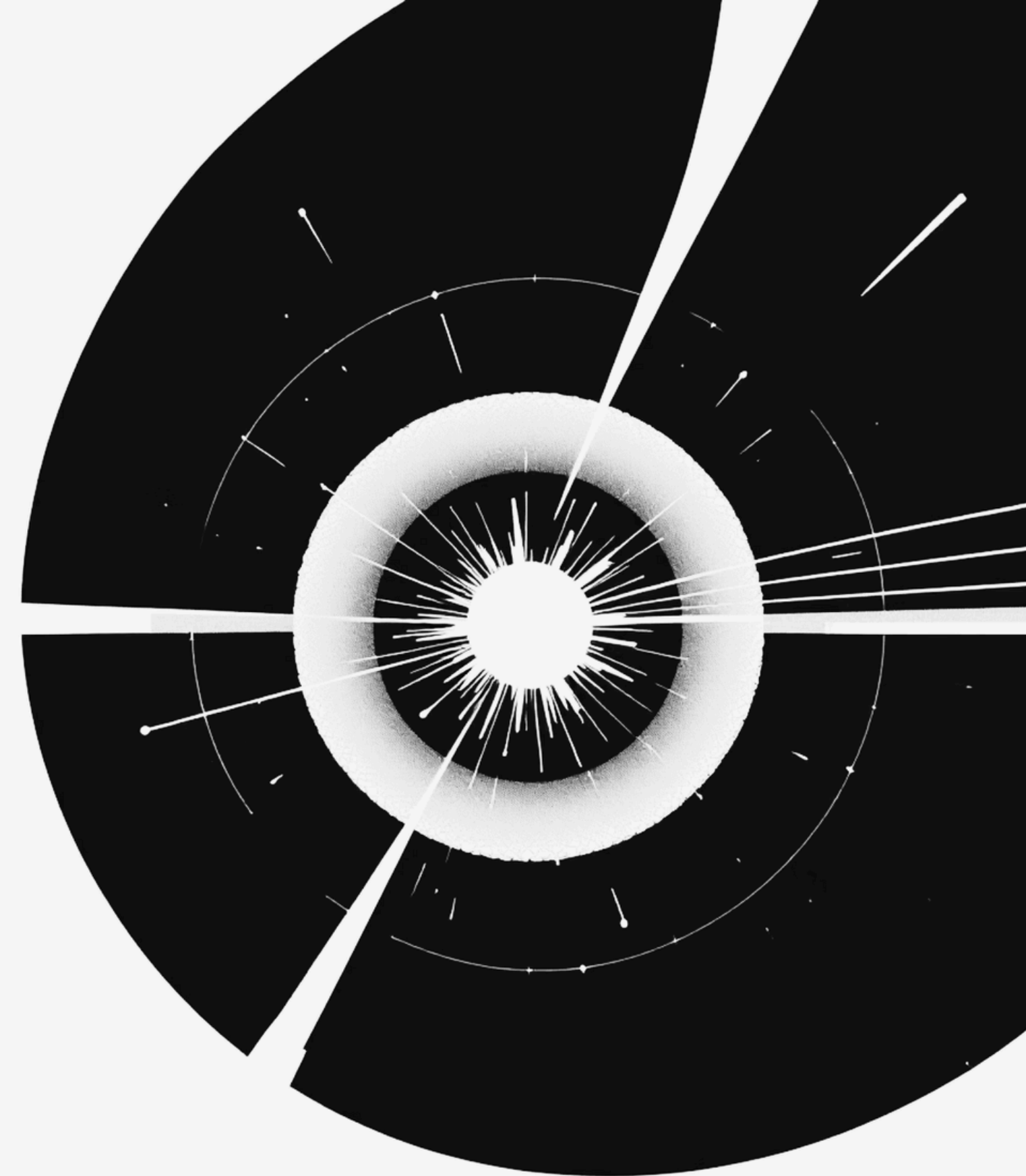
– Ensemble Model

- Identifying relevant models and training to learn weights

– Overall

- Compared models on RMSE for both scaled and unscaled outputs

– Future Scope



Data Collection and Processing

Numerical Stability

Raw fusion data presents extreme challenges:

- Temperatures: 2.5×10^8 - 4×10^8 Kelvin
- Densities: $8 \times 10^{19} \text{ m}^{-3}$
- Power outputs spanning orders of magnitude

We implemented careful scaling strategies:

- 1. Temperature converted to keV units ($1 \text{ keV} \approx 1.16 \times 10^7 \text{ K}$)
- 1. Density normalized to 10^{20} m^{-3} scale
- 1. Power output standardized (zero mean, unit variance)
- 1. Categorical variables one-hot encoded

This prevents numerical overflow, gradient explosions, and the notorious $\beta = \text{NaN}$ problem that happened a lot on our early training attempts.

01

Data cleaning

Removing datapoints with invalid data, missing entries

02

Data categorization

Removing datapoints with invalid data, missing entries

03

Value Scalling

Removing datapoints with invalid data, missing entries

04

Train-Test Split

Removing datapoints with invalid data, missing entries

Understanding the Dataset



Simulated Fusion Experiments

10,000+ experimental runs from tokamak, stellarator, and reversed field pinch configurations



Key Physical Variables

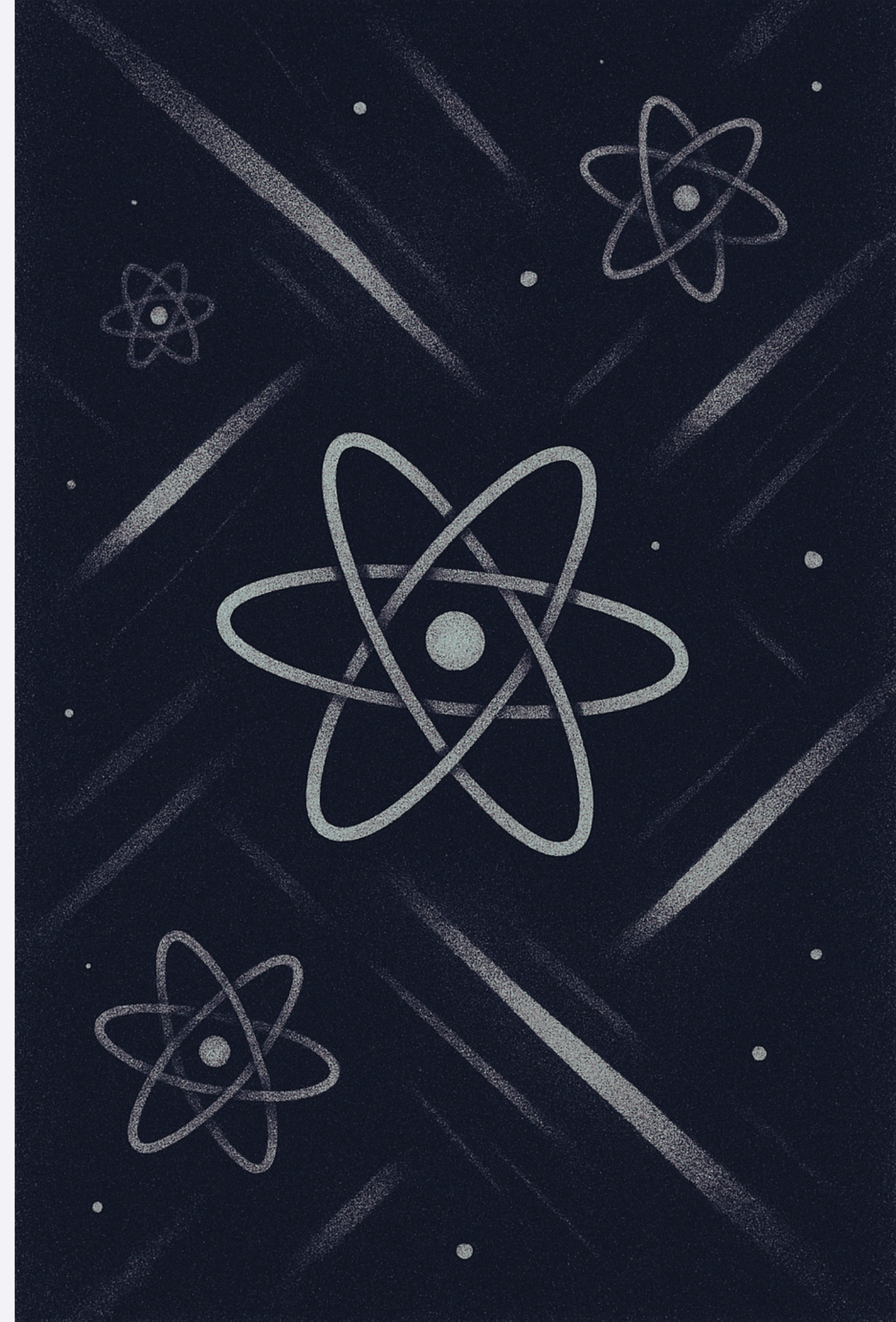
Temperature ($\sim 10^8$ K), fuel density ($\sim 10^{20}$ m⁻³), confinement time, magnetic field properties



Target Variable

Power Output - the critical metric determining fusion reactor viability and energy gain

The dataset captures the complex interplay between plasma conditions, magnetic confinement, and energy output. Each row represents a unique experimental configuration with measurements spanning magnetic field fluctuations, plasma instabilities, fuel composition (deuterium, tritium, D-T mixtures), injection parameters, and resulting power generation metrics.



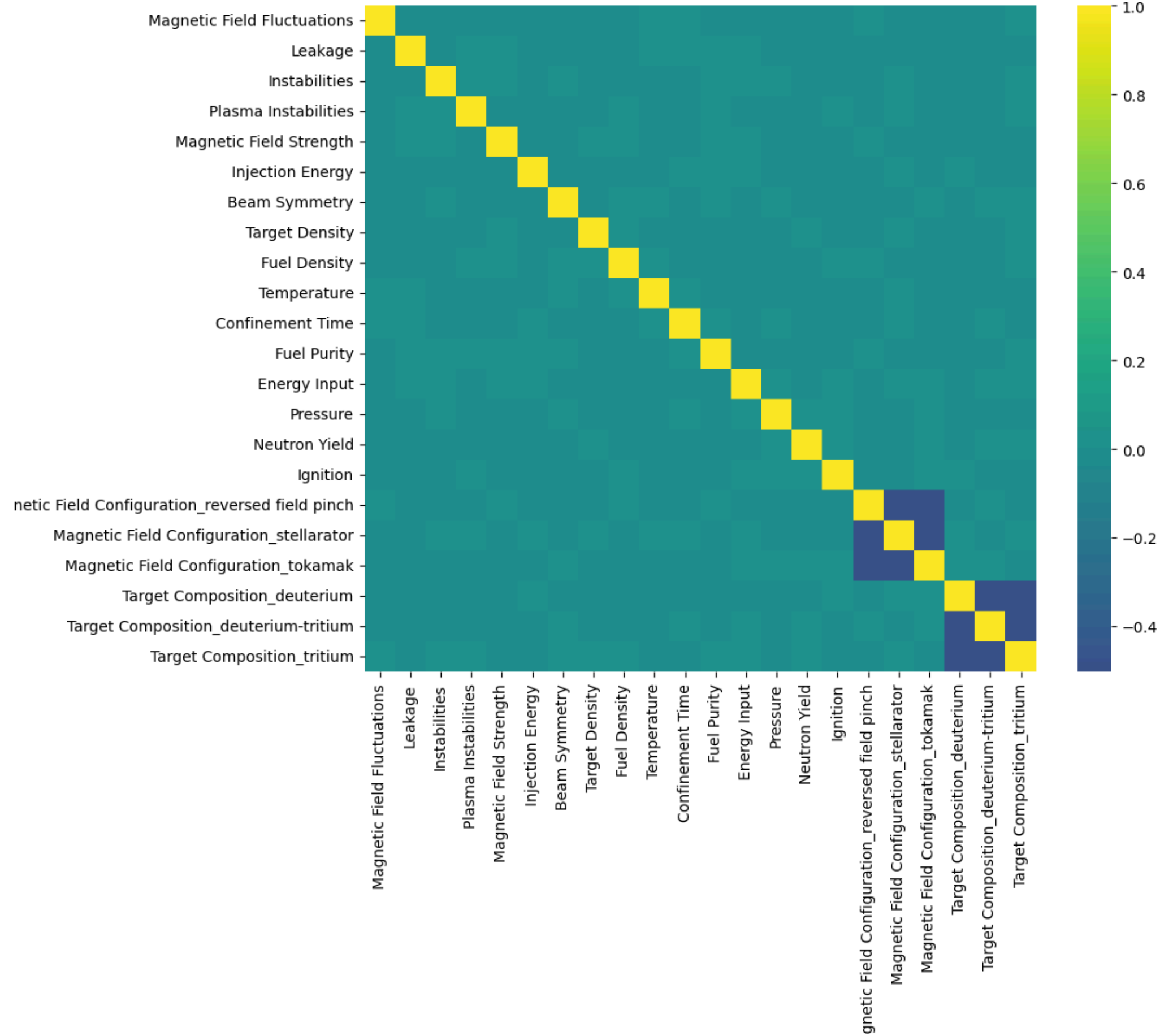
Pre-processed Data

	Magnetic Field Fluctuatio ns	Leakage	Instabilitie s	Plasma Instabilitie s	Magnetic Field Strength	Magnetic Field Configurat ion	Injection Energy	Beam Symmetry	Target Density	Target Compositi on	Fuel Density	Temperat ure	Confineme nt Time	Fuel Purity	Energy Input	Power Output	Pressure	Neutron Yield	Ignition
0	0.037454	0.058078	0.028259	0.015705	9.000578	tokamak reversed	5.713125	0.800007	2.03E+19	deuterium	7.61E+19	4.05E+08	0.842126	99.97129	250.7267	55.32152	6107792	5.44E+10	0
1	0.095071	0.052697	0.045868	0.009551	3.841421	field pinch	9.819548	0.082642	3.05E+19	-tritium	8.74E+19	2.55E+08	0.504637	99.95105	345.6142	22.76796	4921946	8.37E+10	0
2	0.073199	0.035104	0.009922	0.013794	1.467187	stellarator	7.016781	0.176319	5.29E+19	tritium	8.24E+19	2.63E+08	0.357445	99.95883	290.051	49.87294	9798230	8.11E+10	1
3	0.059866	0.049321	0.044684	0.047349	9.277696	tokamak reversed	4.01893	0.833709	5.96E+19	-tritium	9.08E+19	3.1E+08	0.992195	99.99719	436.4911	2.656182	5611293	4.42E+10	1
4	0.015602	0.03651	0.020308	0.088453	4.926347	field pinch	6.145836	0.808161	7.89E+19	deuterium	8.19E+19	3.26E+08	0.648677	99.92705	198.7732	48.09601	8541064	2.25E+10	0

Processed Data

	Magnetic Field Fluctuatio ns	Leakage	Instabilitie s	Plasma Instabilitie s	Magnetic Field Strength	Injection Energy	Beam Symmetry	Target Density	Fuel Density	Temperat ure	...	Energy Input	Pressure	Neutron Yield	Ignition	Magnetic Field Configurat ion_revers ed field pinch	Magnetic Field Configurat ion_stellar ator	Magnetic Field Configurat ion_toka mak	Target Compositi on_deuter ium	Target Compositi on_deuter ium- tritium	Target Compositi on_tritium
0	-1.23499	0.554692	-1.02844	-1.11735	0.997307	0.955365	1.146684	-0.45152	-0.16493	-0.81045	...	-0.26819	-1.57077	-1.20395	1.520914	1	0	0	0	1	0
1	-0.313	1.42875	-0.84351	-1.65532	0.601477	1.681971	-0.98938	1.401749	-1.34374	-0.17407	...	-1.33877	0.375614	0.49373	-0.6575	0	0	1	0	1	0
2	-0.06703	-0.26845	0.141505	1.131538	1.014078	0.490695	-1.35817	0.092743	-1.66735	-0.52422	...	-1.43779	-1.04605	-0.11762	-0.6575	1	0	0	0	1	0
3	1.57454	-1.53827	-0.43164	0.634515	0.040681	-0.42786	0.686534	-1.27192	-0.40627	0.615395	...	-0.11735	-1.47917	1.195435	-0.6575	0	1	0	0	1	0
4	-1.00401	-0.55247	1.03189	-0.09346	1.214694	-1.31351	-1.09362	0.897578	-1.0423	0.229344	...	-0.84624	-0.21546	0.262126	1.520914	1	0	0	0	1	0

Feature Correlation Matrix



Multiple Linear Regression



Regularization

We experimented with applying all the 3 most popular regularization methods: Lasso, Ridge, Elastic Net



Gradient Descent

Temperature ($\sim 10^8$ K), fuel density ($\sim 10^{20} \text{ m}^{-3}$), confinement time, magnetic field properties

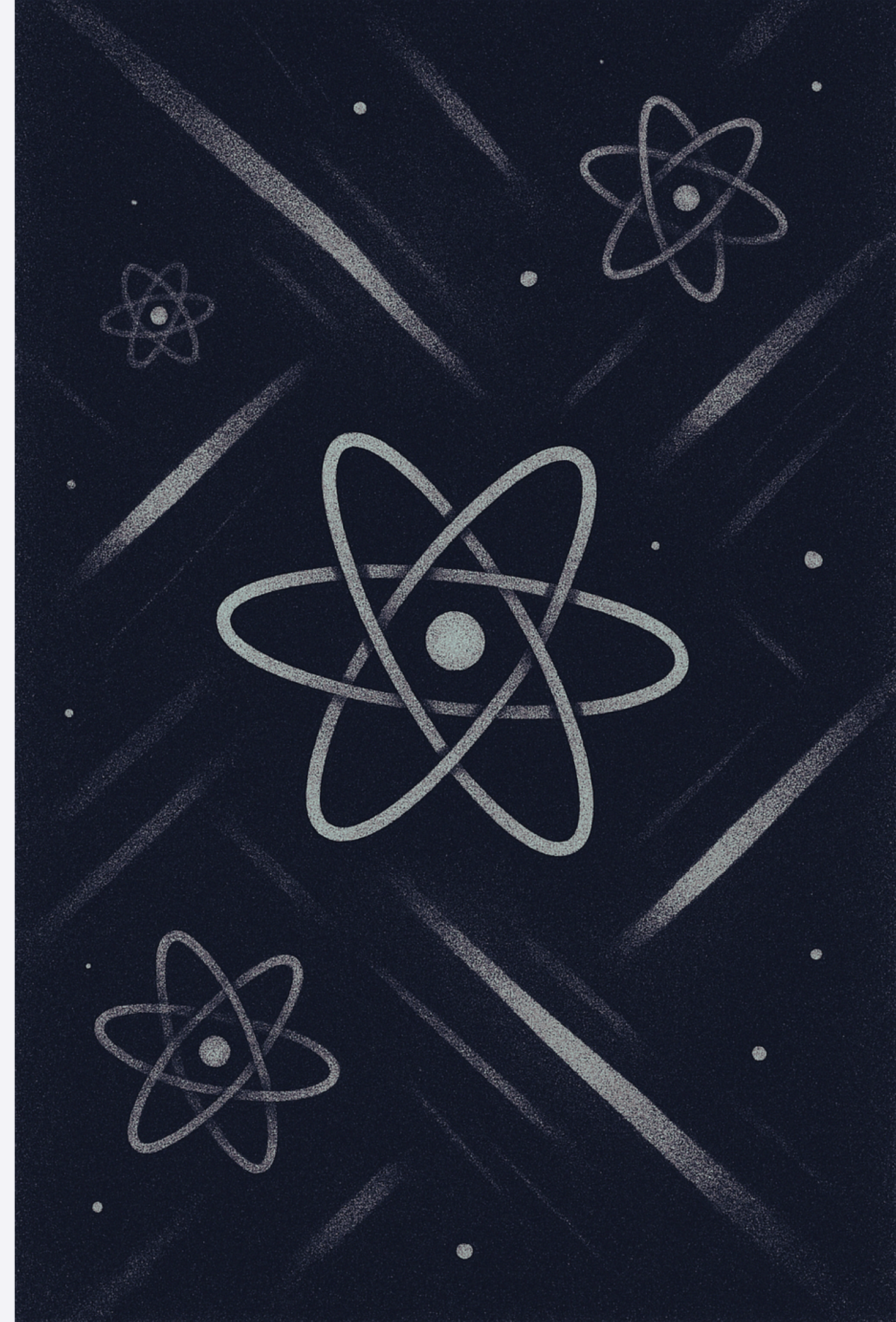


Hyperparameter Tuning

We tuned hyperparameters to optimize model performance and stability, making sure it generalizes well to unseen data.

We explored using grid and random search approaches with cross-validation to semantically search and evaluate performance of different configurations and selected the combination that minimized validation loss.

Exploring the various options, we finally settled on Linear Regression with Elastic Net Regularization and Hyperparameter Parameter Optimization with random search approach with cross-validation, which gave us descent results.



Hyper Parameter Tuning

- For Stochastic Gradient Descent used in Linear Regression without regularization we used the following values of learning rate [0.2, 0.1, 0.01, 0.001]
- The Best SGD results were at the learning rate of 0.001
- For linear regression with regularization learning rate used where [0.001, 0.01, 0.1, 1.0, 10.0]
- Lasso , alpha : 10
- Ridge : 1
- Elastic Net had l1_ratio parameter tuned that is [0.2 , 0.5 , 0.8]
- We had best params for elastic net as alpha 0.1 and 0.8.

Linear Regression

```
Best SGD params: {'eta0': 0.001}  
Best CV MSE (neg): -842.3276264101902
```

```
[SGD Linear Regression] Train results  
MSE: 842.784623698921  
MAE: 25.103507852809173  
R2 : -0.010712367790495891
```

```
[SGD Linear Regression] Test results  
MSE: 833.9515881522408  
MAE: 24.909623954215487  
R2 : -0.013278251275792519
```

Regularized Linear Regression

```
Best Ridge params: {'alpha': 10.0}  
Best Lasso params: {'alpha': 0.1}  
Best ElasticNet params: {'alpha': 0.1, 'l1_ratio': 0.8}
```

```
[Ridge] Train results  
MSE: 833.6396992030836  
MAE: 25.00701735035333  
R2 : 0.000254714463433503
```

```
[Ridge] Test results  
MSE: 823.1259712003325  
MAE: 24.8042197437861  
R2 : -0.0001247752588984241
```

```
[Lasso] Train results  
MSE: 833.7332835873444  
MAE: 25.008395381700776  
R2 : 0.00014248306771902364
```

```
[Lasso] Test results  
MSE: 823.0286988639903  
MAE: 24.804523493353166  
R2 : -6.58621259574943e-06
```

```
[ElasticNet] Train results  
MSE: 833.7103527841605  
MAE: 25.008097762829482  
R2 : 0.0001699829125545449
```

```
[ElasticNet] Test results  
MSE: 823.0306742374445  
MAE: 24.804394584448925  
R2 : -8.986355466866769e-06
```


Neural Networks (MLP)

```
class Net(nn.Module):  
    def __init__(self, in_features):  
        super().__init__()  
        self.layers = nn.Sequential(  
            nn.Linear(in_features, 64), nn.ReLU(),  
            nn.Linear(64, 32), nn.ReLU(),  
            nn.Linear(32, 16), nn.ReLU(),  
            nn.Linear(16, 1)  
        )  
    def forward(self, x): return self.layers(x)
```


Neural Networks

1. Neural Network Model: Built a feed-forward regression network in PyTorch with architecture $[64 \rightarrow 32 \rightarrow 16 \rightarrow 1]$ using ReLU activations.
2. Training Setup: Used mini-batch training with MSE loss and the AdamW optimizer, and evaluated using MSE, MAE, and R^2 metrics.
3. Hyperparameters Tuned: Performed grid search over learning rate $\{1e-3, 5e-4\}$, weight decay $\{1e-4, 1e-5\}$, batch size $\{32, 64\}$, and epochs = 100.
4. Validation Approach: Split training data into train/validation sets and computed validation MSE for each hyperparameter combination.
5. Best Model Selection: Selected the hyperparameter configuration with the lowest validation MSE and retrained the model on the full training set before final testing.

```
grid = {  
    "lr": [1e-3, 5e-4],  
    "weight_decay": [1e-4, 1e-5],  
    "batch_size": [32, 64],  
    "epochs": [100] |  
}
```


Hyper Parameter Tuning In NN

```
Trying 1/8: {'lr': 0.001, 'weight_decay': 0.0001, 'batch_size': 32, 'epochs': 100}
val MSE = 859.255005
Trying 2/8: {'lr': 0.001, 'weight_decay': 0.0001, 'batch_size': 64, 'epochs': 100}
val MSE = 849.348022
Trying 3/8: {'lr': 0.001, 'weight_decay': 1e-05, 'batch_size': 32, 'epochs': 100}
val MSE = 857.555054
Trying 4/8: {'lr': 0.001, 'weight_decay': 1e-05, 'batch_size': 64, 'epochs': 100}
val MSE = 863.888916
Trying 5/8: {'lr': 0.0005, 'weight_decay': 0.0001, 'batch_size': 32, 'epochs': 100}
val MSE = 851.797974
Trying 6/8: {'lr': 0.0005, 'weight_decay': 0.0001, 'batch_size': 64, 'epochs': 100}
val MSE = 846.635498
Trying 7/8: {'lr': 0.0005, 'weight_decay': 1e-05, 'batch_size': 32, 'epochs': 100}
val MSE = 859.478638
Trying 8/8: {'lr': 0.0005, 'weight_decay': 1e-05, 'batch_size': 64, 'epochs': 100}
val MSE = 853.216370
```

Best hyperparameters (by val MSE):

```
lr: 0.0005
weight_decay: 0.0001
batch_size: 64
epochs: 100
```

Validation MSE: 846.635498

[Train]

MSE: 821.5784392755454

MAE: 24.772844475333418

R2 : 0.014719222045923241

[Test]

MSE: 830.7119233224105

MAE: 24.870409397262517

R2 : -0.009341953341803988

Physics Informed Neural Network (PINN)

Nuclear Fusion very much has its roots in real physical laws, making it a good problem to apply PINNs to. This helped us ensure a model that better respects real constraints and relations.

Physics Loss Part (L_{physics})

Enforces steady-state fusion power law. Nudges the Neural Network into the general direction, based on what should be expected, taking into account the physical relationships in the system, more specifically the Boche-Hale Fusion Reactivity Curve.

DataNet Part (L_{data})

Acts just like the loss function in a simple neural network. Deals with all the variables available to us, filling in the gaps of things that we may not have taken into account while modelling the Physics Loss Equations

Hyperparameter λ_{phys}

Controls the strength of physics enforcement. Too low: model ignores physics. Too high: model can't fit data. We have used $\lambda = 0.1$ (more details in the documentation)

Backpropagation and the Gradient Flow

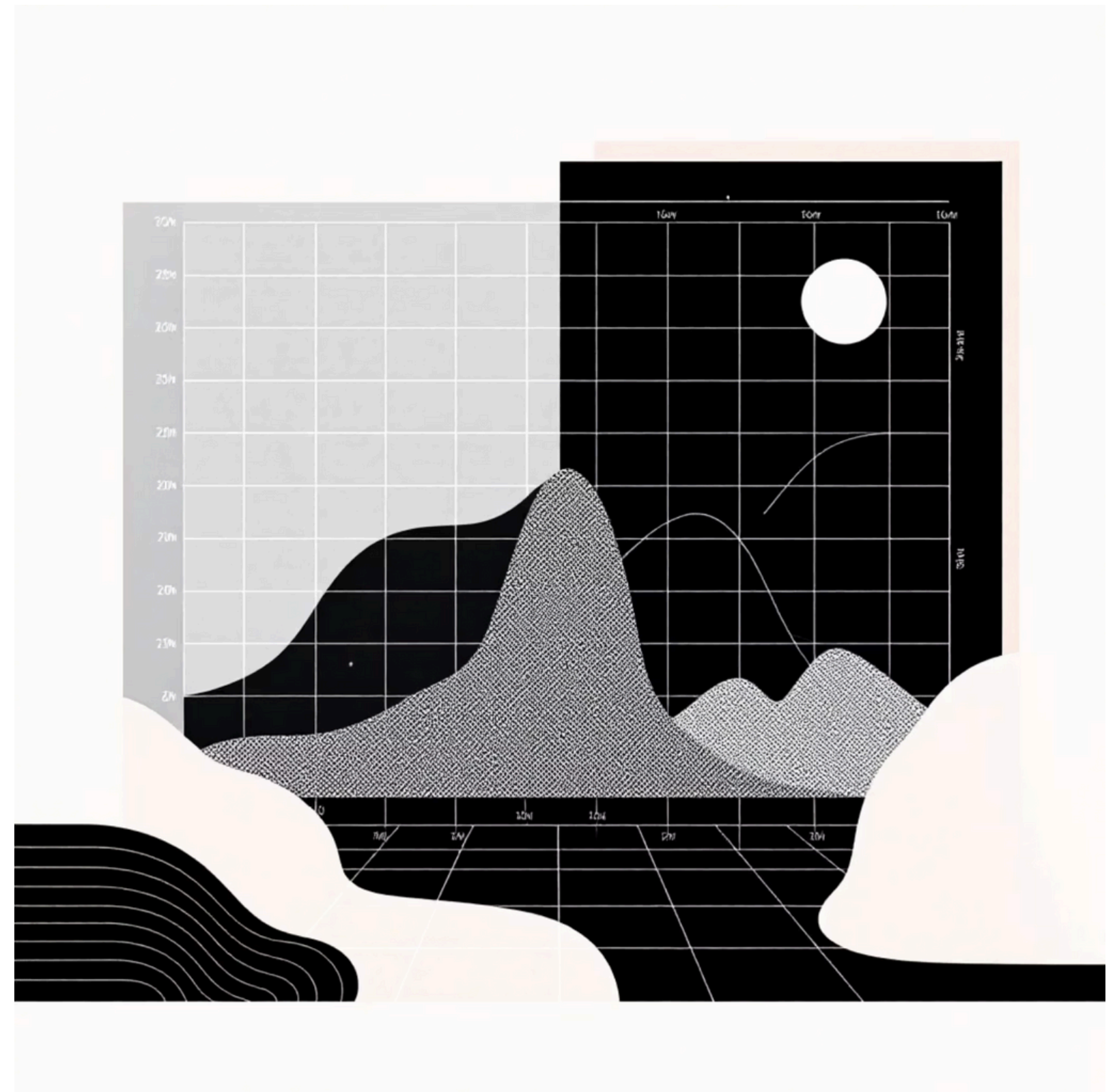
Both loss terms backpropagate simultaneously, updating DataNet weights, $\Phi(T)$ parameters, and β together. This collective optimization allows us to not only get better results, but also have a less rigid Physics Loss term.

Total Loss

$$L = L_{\text{data}} + \lambda_{\text{phys}} \cdot L_{\text{physics}}$$

Key Advantages of PINN Approach:

- **Physical Consistency:** Predictions respect fundamental fusion scaling laws $P \propto n^2 \Phi(T)$, preventing unphysical extrapolations
- **Interpretability:** Learned β and $\Phi(T)$ curve have clear physical meanings, enabling scientific insight beyond black-box predictions
- **Generalization:** Physics constraints act as regularization, improving performance on out-of-distribution reactor configurations



The Physics Behind the Fusion Reaction

The major physical insights into Fusion Reactions' Power Relation that we were able to find are:



Density Dependence

Nuclear Fusion is second order, ie, the reaction scales with n^2 . Makes sense as reaction rate depends on collisions between fuel ions. Higher density means more particles, meaning more collisions.



Temperature Reactivity $\Phi(T)$

Actual Temperature relation deals with complex Quantum Mechanical Relations, so we were not able to use it here. There is a widely adopted simplification of this relation, called the Boche-Hale Fusion Reactivity Curve.

$$\underline{\phi(T)} = T^2 \cdot \exp\left(-\frac{b}{\sqrt{T}}\right)$$



Proportionality Constant β

Although there are other parameters like magnetic field strength, we could not find any better expression to use in the physics loss function, so we finally settled on having β , the proportionality constant as the final parameter in the Physics Loss expression.

This relation both utilizes results from years of plasma physics research while remaining flexible enough to adapt to real experimental conditions through learned parameters.

$$P(n, T) \propto \phi(T) \cdot n^2 \longrightarrow P(n, T) = \underbrace{\beta} \cdot \underbrace{\phi(T)} \cdot \underbrace{n^2}$$



PINN Architecture: Three Learning Components

- The loss function is still not ready as we still have 2 unknown in the equation.
- $\mathbf{b}$ in the $\Phi(\mathbf{T})$ relation and β , the proportionality constant.
- We have 2 unknown parameters in an equation we need to fit, so we decided to go with a simple regression to find the exact physics loss function.

Results: PINN vs Standard Neural Network

Performance Metrics

28.67

PINN Train RMSE

Strong fit to training data without overfitting

30.26

PINN Test RMSE

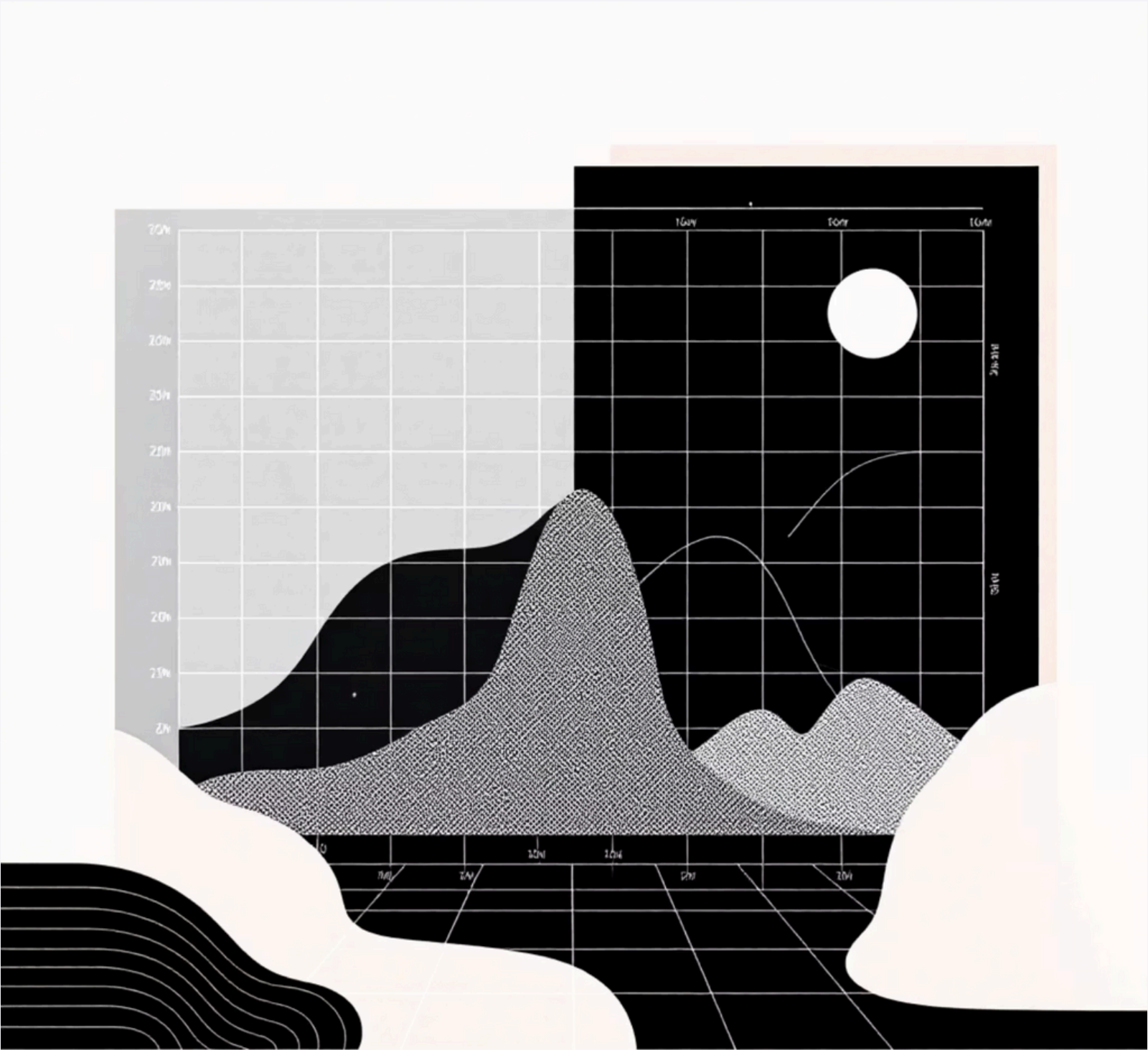
Excellent generalization to unseen configurations

0.684

Learned β Value (scaled)

Physically reasonable scaling constant

The PINN demonstrates stable training with no NaN explosions, converging to a physically interpretable solution. The modest gap between train and test RMSE indicates good generalization rather than memorization.



PINN model architecture

Bosch–Hale Fusion Reactivity Model

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import math # Added this line

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(42)

# --- pull the raw columns we need for physics (from the original, untransformed splits) ---
# We already have: X_train, X_test, y_train, y_test from Section 6
# Make sure the names exist as in CSV:
T_col = "Temperature"      # [K]
n_col = "Fuel Density"     # [m^-3], ~1e20
assert T_col in X_train.columns and n_col in X_train.columns, "Missing physics columns."

# Prepare tensors: transformed features for the NN + raw T, n for physics
Xtr = torch.tensor(X_train_t, dtype=torch.float32)
Xte = torch.tensor(X_test_t, dtype=torch.float32)
ytr = torch.tensor(y_train.values, dtype=torch.float32).view(-1, 1)
yte = torch.tensor(y_test.values, dtype=torch.float32).view(-1, 1)

# Raw physical inputs (match row order from the same split!)
Ttr = torch.tensor(X_train[T_col].values, dtype=torch.float32).view(-1, 1)
ntr = torch.tensor(X_train[n_col].values, dtype=torch.float32).view(-1, 1)
Tte = torch.tensor(X_test[T_col].values, dtype=torch.float32).view(-1, 1)
nte = torch.tensor(X_test[n_col].values, dtype=torch.float32).view(-1, 1)

class PINNDataset(Dataset):
    def __init__(self, X, y, T, n):
        self.X, self.y, self.T, self.n = X, y, T, n
    def __len__(self):
        return self.X.shape[0]
    def __getitem__(self, i):
        return self.X[i], self.y[i], self.T[i], self.n[i]

train_ds = PINNDataset(Xtr, ytr, Ttr, ntr)
test_ds = PINNDataset(Xte, yte, Tte, nte)

train_loader = DataLoader(train_ds, batch_size=256, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=512, shuffle=False)
```

```
# --- simple MLP regressor for Power Output ---
class MLP(nn.Module):
    def __init__(self, in_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 128),
            nn.GELU(),
            nn.Linear(128, 64),
            nn.GELU(),
            nn.Linear(64, 1)
        )
    def forward(self, x):
        return self.net(x)

# --- steady-state physics:  $P \approx \beta * (n^2) * \phi(T)$  ---
# Use a smooth surrogate for D-T reactivity  $\phi(T)$  (Bosch-Hale-like bell curve, unitless after scaling).
@torch.no_grad()
def _norm_constants(T, n):
    # scale to keep numbers ~0(1) for stable training
    T_scale = max(1.0, torch.quantile(T, 0.95).item()) # K
    n_scale = max(1.0, torch.quantile(n, 0.95).item()) # m^-3
    y_scale = max(1.0, torch.quantile(ytr, 0.95).item()) # power units
    return T_scale, n_scale, y_scale

T_scale, n_scale, y_scale = _norm_constants(Ttr, ntr)

class Physics:
    def __init__(self, T_scale, n_scale):
        self.T_scale = T_scale
        self.n_scale = n_scale
        # constants for a smooth  $\phi(T)$  (not exact physics – safe surrogate)
        self.c1 = 1.0 # will be renormalized; keep 0(1)
        self.c2 = 4.5 # controls peak location
        self.eps = 1e-7
```

Additional Applications in Chemical Engineering

Based on our learnings working with this project, we have identified some other areas in Chemical Engineering where we could apply a similar approach:

1. Reactor Modeling:

- a. PINNs can model CSTR/PFR behavior by combining reaction kinetics with plant data.

2. Heat & Mass Transfer:

- a. Solve conduction/diffusion equations for exchangers, catalyst beds, and membranes.

3. Process Control & Digital Twins:

- a. Create physics-consistent predictive models for safe and efficient plant operation.

4. Kinetic Parameter Estimation:

- a. Learn reaction constants (like β) directly from limited experimental data.

5. Multiphysics Simulations:

- a. Accelerate modeling of coupled transport–reaction systems (e.g., CO₂ capture, electrochemical

Future Scope

- Find an alternative PINN relation using symbolic regression
- Improve PINN Physics Loss Equation
- Inverse PINN (treat b and β as trainable scalars)
- A better analysis can be done using time variant data
- Make an Ensemble model, mixing and matching different models